

BIT4

Instruction Set Reference

35 Instructions • Complete Reference • With Examples

09.06.2026

Overview

BIT4 is a 4-bit stack-based CPU with 35 instructions across 7 categories. All ALU operations read operands from the stack and push results back. MOV loads values into registers directly. PUSH/POP move data between registers and the stack.

Category	Count	Instructions
Data Movement	9	MOV AX, MOV BX, MOV AX/BX, PUSH AX/BX, POP AX/BX
Memory	5	LOAD, STORE, LOADB, STOREB, LOADBI
ALU — Logical	4	AND, OR, XOR, NOT
ALU — Arithmetic	6	ADD, SUB, INC, DEC, SHL, SHR
Compare	2	CMP AX, CMP BX
Control Flow	5	JMP, JZ, JNZ, CALL, RET
VPU	1	PRINT
System	3	NOP, HALT, (labels)

1. Data Movement

Load immediate values into registers, or copy between registers and the stack.

Mnemonic	Encoding	Operation	Stack Effect
MOV AX, xxxx	1100xxxx	Load 4-bit immediate into AX register	None
MOV BX, xxxx	1101xxxx	Load 4-bit immediate into BX register	None
MOV AX, BX	11001101	Copy BX register value into AX	None
MOV BX, AX	11011100	Copy AX register value into BX	None
PUSH AX	11000000	Push AX value onto stack top	+1
PUSH BX	11010000	Push BX value onto stack top	+1
POP AX	11100000	Pop stack top into AX register	-1
POP BX	11110000	Pop stack top into BX register	-1

Examples

```
mov ax,1010 ; AX = 1010
mov bx,ax ; BX = 1010 (copy from AX)
push ax ; stack: [1010]
pop bx ; BX = 1010, stack: []
```

2. Memory Operations

BIT4 has 256 bytes of RAM across 16 banks of 16 bytes each. LOAD/STORE access the active bank. LOADB/STOREB access any bank directly. LOADBI uses BX as a dynamic address pointer.

Mnemonic	Encoding	Operation	Stack
LOAD 0x??	0110xxxx	Read active bank RAM[addr] → push to stack	+1
STORE 0x??	0111xxxx	Pop stack → write to active bank RAM[addr]	-1
LOADB bank,addr	special	Read specific bank addr directly → push to stack	+1
STOREB bank,addr	special	Pop stack → write to specific bank addr directly	-1
LOADBI bank	special	Read bank[BX] → push to stack (indirect)	+1

Examples

```

mov ax,1010
push ax
store 0000 ; RAM[0] = 1010
load 0000 ; stack: [1010]
storeb 2,0000 ; write to bank 2 addr 0
loadb 2,0000 ; read from bank 2 addr 0
mov bx,0011 ; BX = 3 (address pointer)
loadbi 4 ; read bank 4[3]

```

Hardware ports 0x04-0x0F are intercepted — writing to them triggers VPU, keyboard, bank switch, or display commands instead of normal RAM write.

3. ALU — Logical Operations

All logical operations are binary (except NOT which is unary). They pop operands from the stack and push the result back. The ZeroFlag is set if the result is 0000.

Mnemonic	Opcode	Operation	Type	ZeroFlag
AND	0000	Bitwise AND of top two stack values	Binary	Yes
OR	0001	Bitwise OR of top two stack values	Binary	Yes
XOR	0010	Bitwise XOR of top two stack values	Binary	Yes
NOT	0011	Bitwise NOT of stack top	Unary	Yes

XOR as comparison — classic pattern:

```
load 0008 ; read keyboard
mov ax,0101
push ax
xor ; result = 0000 only if key == 0101
pop ax ; clean stack
jz key_match ; jump if equal
```

4. ALU — Arithmetic Operations

Arithmetic operations work on 4-bit values (0-15). All values wrap around — $1111 + 0001 = 0000$. INC, DEC, SHL, SHR are unary (one operand). ADD, SUB are binary.

Mnemonic	Opcode	Operation	Type	Notes
ADD	0100	4-bit addition	Binary	Wraps at 1111
SUB	0101	4-bit subtraction	Binary	Wraps at 0000
INC	1010	Increment top of stack	Unary	$1111 \rightarrow 0000$
DEC	1011	Decrement top of stack	Unary	$0000 \rightarrow 1111$
SHL	1000	Shift left (multiply x2)	Unary	MSB lost
SHR	1001	Shift right (divide /2)	Unary	LSB lost

Counter loop example:

```
mov ax,0000
push ax
store 0000 ; counter = 0
loop:
load 0000
inc
store 0000 ; counter++
load 0000
mov ax,1111
push ax
xor
pop ax
jz done ; stop at 15
jmp loop
done:
```

5. Compare Instructions

CMP compares a register against an immediate value and sets the ZeroFlag. Unlike XOR, CMP does not modify the stack — clean and simple.

Mnemonic	Encoding	Operation	Stack Effect
CMP AX, xxxx	10111100	Compare AX with immediate — set ZeroFlag if equal	None
CMP BX, xxxx	10111101	Compare BX with immediate — set ZeroFlag if equal	None

CMP vs XOR comparison:

Old way with XOR (stack gets dirty):

```
load 0008
mov ax,0101
push ax
xor ; stack: [result]
pop ax ; must clean stack!
jz match
```

New way with CMP (stack stays clean):

```
load 0008
pop ax ; move to register
cmp ax,0101 ; compare – stack untouched
jz match ; clean!
```

6. Control Flow

Labels are declared with a colon suffix. Labels are resolved at compile time and do not generate machine code. CALL/RET use a separate call stack.

Mnemonic	Encoding	Condition	Action
label:	LABEL	—	Declare a jump target — no machine code generated
JMP label	11111111	Always	PC = label address unconditionally
JZ label	11111110	ZeroFlag == true	Jump if last ALU/CMP result was zero
JNZ label	11111011	ZeroFlag == false	Jump if last ALU/CMP result was non-zero
CALL label	11111101	Always	Push PC+1 to call stack, jump to label
RET	11111100	Always	Pop call stack, return to caller

Subroutine example:

```

call draw_char ; push return address, jump
jmp done
draw_char:
mov ax,0000
push ax
store 0009 ; ROM page 0
mov ax,0000
push ax
print ; draw A
ret ; return to caller
done:
halt

```

7. VPU & System Instructions

VPU

Mnemonic	Encoding	Operation
PRINT	11110111	Pop stack → convert to char ID → look up in active ROM page → draw 4x5 bitmap at cursor

```
mov ax,0000
push ax
store 0009 ; select ROM page 0 (A-P)
mov ax,0000
push ax
print ; draw character A (ID 0)
```

System

Mnemonic	Encoding	Operation	Use Case
NOP	00000001	No operation — consumes one CPU cycle	Timing delays, padding
HALT	11111110	Stop CPU execution immediately	Program end, game over

```
nop ; wait one cycle
nop ; wait one more
halt ; stop execution
```

Quick Reference — All 35 Instructions

#	Mnemonic	Category	Stack	ZF
1	MOV AX, xxxx	Data Movement	None	—
2	MOV BX, xxxx	Data Movement	None	—
3	MOV AX, BX	Data Movement	None	—
4	MOV BX, AX	Data Movement	None	—
5	PUSH AX	Data Movement	+1	—
6	PUSH BX	Data Movement	+1	—
7	POP AX	Data Movement	-1	—
8	POP BX	Data Movement	-1	—
9	LOAD 0x??	Memory	+1	—
10	STORE 0x??	Memory	-1	—
11	LOADB bank,addr	Memory	+1	—
12	STOREB bank,addr	Memory	-1	—
13	LOADBI bank	Memory	+1	—
14	AND	ALU Logical	-1	Yes
15	OR	ALU Logical	-1	Yes
16	XOR	ALU Logical	-1	Yes
17	NOT	ALU Logical	0	Yes
18	ADD	ALU Arithmetic	-1	Yes
19	SUB	ALU Arithmetic	-1	Yes
20	INC	ALU Arithmetic	0	Yes
21	DEC	ALU Arithmetic	0	Yes
22	SHL	ALU Arithmetic	0	Yes
23	SHR	ALU Arithmetic	0	Yes
24	CMP AX, xxxx	Compare	None	Yes
25	CMP BX, xxxx	Compare	None	Yes
26	JMP label	Control Flow	None	—
27	JZ label	Control Flow	None	—
28	JNZ label	Control Flow	None	—
29	CALL label	Control Flow	None	—
30	RET	Control Flow	None	—
31	PRINT	VPU	-1	—
32	NOP	System	None	—

#	Mnemonic	Category	Stack	ZF
33	HALT	System	None	—
34	label:	System	None	—
35	LOADBI bank	Memory	+1	—

BIT4 — Instruction Set Reference — 09.06.2026

35 instructions — 4-bit stack-based CPU — hand-implemented in C#